

# Week 02: Data and structural recursion

Functional Programming in Lean

# Learning goals

- ▶ Define a sum type and a structure
- ▶ Use pattern matching to cover constructors
- ▶ Recognize a recursive call on a smaller value

# Central idea

A datatype lists the shapes of its values. A recursive function handles each shape.

# Lean example

```
inductive Suit where
  | clubs | diamonds | hearts | spades

def isRed : Suit → Bool
  | .hearts | .diamonds => true
  | .clubs | .spades => false
```

# What to notice

- ▶ Card groups a suit and a rank
- ▶ List Card is either empty or a head followed by a tail
- ▶ A recursive call on the tail supplies the termination argument

# Live coding

1. Complete `isRed` by cases
2. Define `countRed` on a card list
3. Add a two-card `#eval` example

# Pause and predict

Which case of `countRed` handles the empty deck?

# Exercise connection

Open `exercises/Week02.md` after the lecture.

The complete example is `Course/Week02.lean`; the fill-in version is `Templates/Week02.lean`.